H

实验：xv6 项目

注意：这些项目正在陆续添加到 `https://github.com/remzi-arpacidusseau/ostep-projects`，那里有项目说明和一个简单的测试框架。如果你感兴趣，一定要去看看。

本章给出一些和 xv6 内核有关的项目思路。这个内核可以从 MIT 获取，拿来折腾非常好玩；做这些项目还能让课堂上的内容和项目联系得更直接。这些项目（也许最开始的一两个除外）通常两人一组完成，这样盯着内核看这项苦差事就会轻松一点。

H.1 入门项目

入门项目是给 xv6 添加一个简单的系统调用（system call）。可以有很多变体，比如一个统计已经发生了多少次系统调用的系统调用（每个系统调用一个计数器），或者其他收集信息的调用。学生会学到系统调用实际上是怎样发生的。

H.2 进程和调度

学生要实现一个比默认的时间片轮转（Round-Robin）更复杂的调度程序（scheduler）。可以有很多变体，比如彩票调度（lottery scheduling）程序或多级反馈队列（Multi-level Feedback Queue）。学生会学到调度程序实际上是怎么工作的，以及上下文切换（context switch）是怎样发生的。另外还有一个小要求：让学生想办法让进程（process）在退出时返回正确的错误码，并能通过 `wait()` 系统调用拿到这个错误码。

H.3 虚拟内存入门

基本思路是添加一个新的系统调用：给定一个虚拟地址（virtual address），返回转换后的物理地址（或者报告这个地址无效）。这能让学生看到虚拟内存（virtual memory）系统是怎样建立页表（page table）的，又不用做太多费力的工作。另一个变体研究怎样改造 xv6，让对空指针（null pointer）解引用真正引发一个故障。

H.4 写时复制映射

这个项目给 xv6 添加一种轻量级的 `fork()`，叫作 `vfork()`。这个新调用不是简单地复制映射，而是给共享的页建立写时复制（copy-on-write）映射。访问这样的页时，内核必须创建一份真正的副本，并相应地更新页表。

H.5 内存映射

另一个虚拟内存项目是添加某种形式的内存映射文件（memory-mapped file）。最简单的做法大概是从可执行文件中惰性调入代码页（lazy page-in）；更完整的做法是实现一个 `mmap()` 系统调用，以及在解引用时从磁盘把页调入所需的全部基础设施。

H.6 内核线程

这个项目研究怎样给 xv6 添加内核线程（kernel thread）。`clone()` 系统调用的工作方式和 `fork` 很像，但使用同一个地址空间（address space）。学生要想办法实现这样一个调用，从而创建一个真正的内核线程。学生还应该在它之上构建一个小型线程库（thread library），提供简单的锁。

H.7 高级内核线程

学生要在自己的内核线程之上构建一个完整的线程库，加入不同类型的锁，比如自旋锁（spin lock）和在处理器不可用时睡眠的锁，以及条件变量（condition variable）。所需的内核支持也要一并加上。

H.8 基于区段的文件系统

这第一个文件系统（file system）项目给基本的文件系统添加一些简单的功能。对于 `EXTENT` 类型的文件，学生要修改 inode，让它存储区段（即指针和长度对），而不是只存指针。这个项目可以作为文件系统的一个比较轻松的入门。

H.9 快速文件系统

学生要把 xv6 的基本文件系统改造成伯克利快速文件系统（FFS）。学生要实现一个新的 `mkfs` 工具，引入块组（block group）和新的块分配策略，并实现大文件的例外规则。这样能更深入地理解文件系统的基本工作原理。

H.10 日志文件系统

学生要给 xv6 添加一个最基本的日志层。每次写文件时，日志文件系统（journaling file system）都把所有被弄脏的块攒成一批，把它们待完成的更新记录写到磁盘上的日志里；只

 © 2008–23, ARPACI-DUSSEAU

有在这之后, 才原地修改这些块。学生要引入崩溃点, 并展示文件系统总能恢复到一致状态, 以此证明自己系统的正确性。

H.11 文件系统检查程序

学生要为 xv6 文件系统实现一个简单的文件系统检查程序（file system checker）。学生会学到是什么让文件系统保持一致，以及到底该怎样检查一致性。

 © 2008–23, ARPACI-DUSSEAU